PI 2 **Proyecto intermodular II**

Git

Proyecto Intermodular II | Git


Git: Del Caos a la Armonía
Orquestando la Colaboración en Equipos de Alto Rendimiento
NotebookLM

El Conflicto: La Fricción Inevitable del Crecimiento



Un conflicto ocurre cuando Git no puede fusionar cambios automáticamente porque las mismas líneas han sido modificadas de forma diferente.

No son errores, sino una protección de Git para evitar sobrescrituras accidentales. Surgen en operaciones como `merge`, `rebase`, o `pull`.

NotebookLM

Anatomía del Caos: ¿Dónde Nacen los Conflictos?



Múltiples Ramas Locales

Un desarrollador edita las mismas secciones de código en ramas paralelas.
Frecuencia: Alta en experimentos locales.



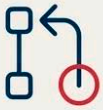
Dos Desarrolladores, Mismo Archivo

El escenario clásico. Dos desarrolladores modifican el mismo módulo en ramas diferentes.
Frecuencia: Muy alta en proyectos monolíticos.



Actualización desde Main

Tu rama de `feature` choca con nuevos commits de la rama principal al intentar un `pull` o `rebase`.
**Frecuencia: Alta en proyectos de rápido avance.*



Revisión de Pull Requests

La rama del PR contiene modificaciones que ya se han integrado en `main` por otra vía.
**Frecuencia: Media-alta en repositorios con muchos contribuidores.*

NotebookLM


El Arsenal Táctico
Herramientas de Precisión para Controlar el Flujo
Para cada tipo de fricción, existe una herramienta diseñada para restaurar el orden.
No se trata de memorizar comandos, sino de entender su propósito táctico.
NotebookLM

Pausa y Reanuda: El Arte de `git stash`

Guarda cambios sin hacer commit para dejar tu directorio de trabajo limpio. Es tu "guardado temporal" cuando las interrupciones son inevitables.

****Casos de Uso**:**

- Aparece un bug urgente y debes cambiar de rama.
- Necesitas hacer `git pull` pero tienes cambios sin 'commitear'.
- Estás explorando una idea y quieres guardarla sin subirla.

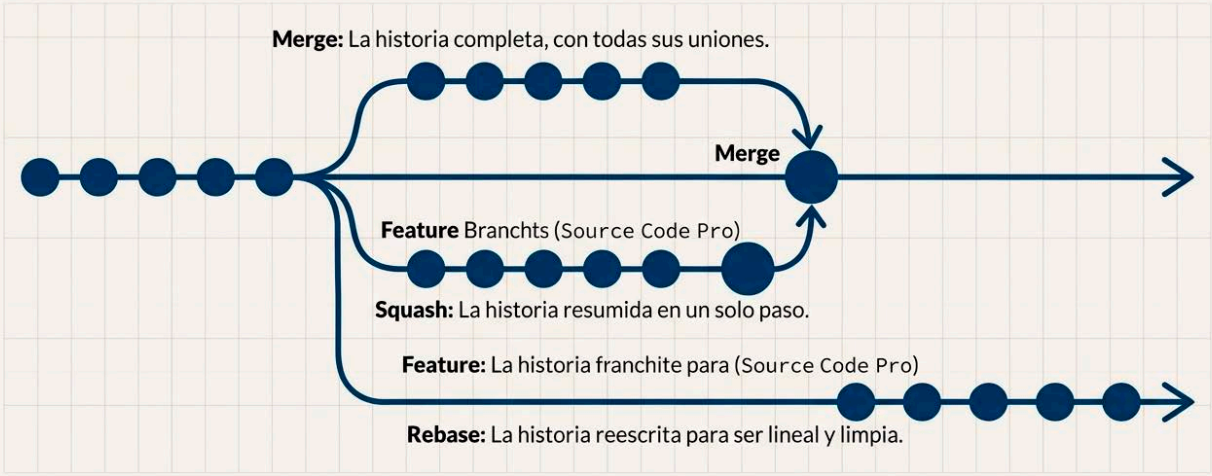
Comandos Esenciales:**

```
`git stash save "Mensaje"`
`git stash list`
`git stash pop`
`git stash apply`
```



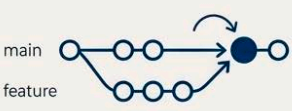
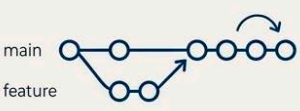
Tres Filosofías de Fusión: `merge`, `rebase` y `squash`

Unir ramas es el corazón de la colaboración. La estrategia que elijas define la historia de tu proyecto. Cada una tiene un propósito y un impacto diferente en la claridad y la trazabilidad del código.



NotebookLM

Elige tu Estrategia: Impacto en el Historial

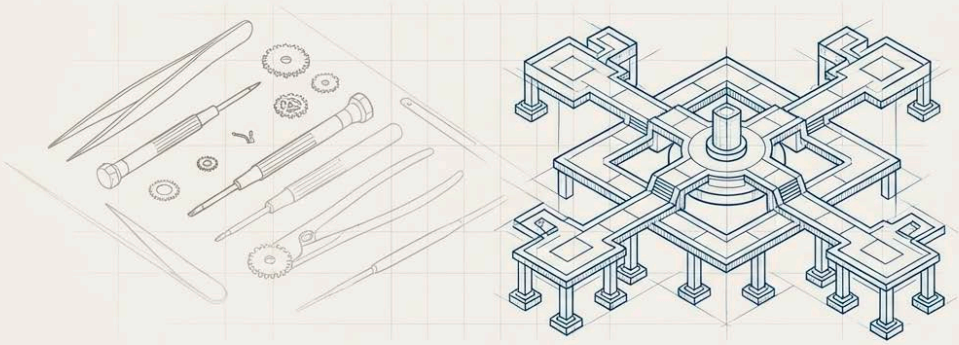
Característica	Merge	Squash Merge	Rebase
Historial	Conserva el historial exacto. "Ruidoso" pero auditable.	Combina commits en uno solo. Muy limpio, pero pierde granularidad.	Lineal y limpio. Reescribe la historia (cambia IDs de commit).
Commit de Fusión	✔ Sí	✖ No	✖ No
Ideal Para	Equipos grandes, auditoría, mantener el contexto histórico.	Pull Requests de features pequeñas para mantener `main` limpio.	Sincronizar tu rama con `main` antes de un PR, trabajo personal.
			

NotebookLM


Domina `git reset`: ¿Soft, Mixed o Hard?
reset --soft
HEAD
STAGING
Mueve HEAD, pero tus cambios se quedan en el área de staging, listos para un nuevo commit.
reset --mixed (Opción por defecto)
HEAD
STAGING
DIRECTORIO DE TRABAJO
Mueve HEAD y saca tus cambios del staging, pero los mantiene en tu directorio de trabajo como modificaciones.
reset --hard
HEAD
STAGING
DIRECTORIO DE TRABAJO
Mueve HEAD y descarta permanentemente TODOS los cambios, tanto del staging como de tu directorio de trabajo.
¡CUIDADO CON EL HISTORIAL REMOTO!
Si los commits ya están subidos (`pushed`), necesitarás un `force push`, lo cual es peligroso en ramas compartidas.
NotebookLM

La Estrategia Maestra

Diseñando el Flujo de Trabajo del Equipo



Las herramientas son inútiles sin un plan. Una estrategia de ramificación es el acuerdo de equipo que define cómo se crea, gestiona y fusiona el código para garantizar una base estable y desplegable.

NotebookLM

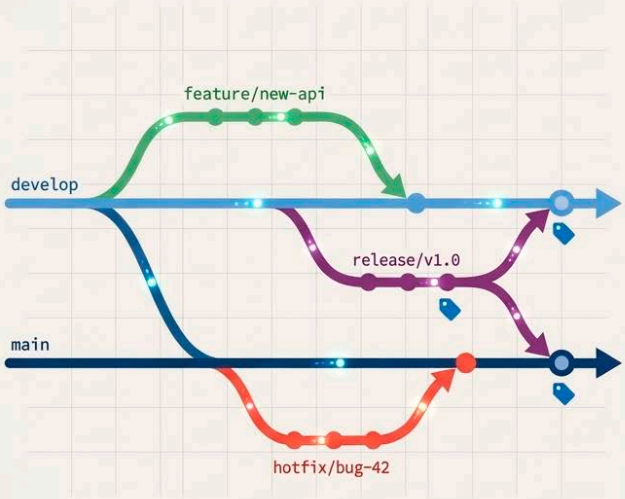
Git Flow: La Orquesta Sinfónica para Releases Planificados

Concepto: Un modelo robusto y estricto, ideal para proyectos con ciclos de lanzamiento definidos y largos (mensuales, trimestrales).

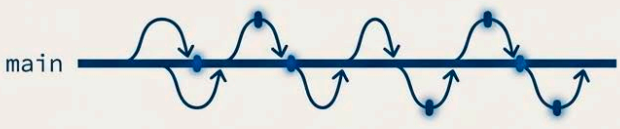
Ramas Clave:

- `main`: Código de producción, históricamente estable.
- `develop`: Integración de features completadas.
- `feature/*`: Desarrollo de nuevas características.
- `release/*`: Preparación del lanzamiento, pruebas finales.
- `hotfix/*`: Arreglos críticos directamente desde `main`.

Ventaja: Estructura muy clara. Aislamiento entre versiones.
Desventaja: Complejo. Alto riesgo de conflictos en ramas de larga duración.



NotebookLM



Trunk-Based Development: El Pulso Rápido de la Integración Continua

Concepto: Un modelo simplificado para equipos que despliegan código varias veces al día (CI/CD).

Reglas Clave:

- Una única rama principal (`main`) siempre en estado desplegable.
- Ramas de `feature` de muy corta duración (horas o un par de días).
- Fusiones constantes a `main` a través de Pull Requests pequeños.

Ventaja: Bajo riesgo de conflictos. Historial limpio y fácil de seguir.

Requisito Clave: Uso de Banderas de Características (***Feature Flags***) para ocultar trabajo incompleto.

NotebookLM

Desplegar sin Miedo: El Poder de las Banderas de Características



****Tipos de Banderas**:**

- **Release**: Ocultan una *feature* grande hasta su lanzamiento.
- **Experiment**: Para pruebas A/B en segmentos de usuarios.
- **Ops**: Para desactivar funciones en picos de tráfico (*Kill Switch*).
- ***Permission***: Para habilitar funciones Premium según el plan del usuario.

***Mecanismo:** El código nuevo se envuelve en una condición que comprueba el estado de una bandera. Esto permite fusionar a `main` de inmediato, incluso si está incompleto.

```
// Código nuevo y antiguo conviven en producción
if (featureFlagService.isEnabled("nueva_interfaz_v2")) {
    mostrarNuevaInterfaz(); // Se activa solo si la bandera es ON
} else {
    mostrarInterfazAntigua(); // Se ejecuta si la bandera es OFF
}
```

NotebookLM


Los Rituales de la Armonía
Gobernanza y Calidad como Práctica Diaria
Las mejores estrategias fallan sin disciplina. Los rituales de equipo son los procesos que convierten un buen flujo de trabajo en un sistema de entrega de valor predecible y de alta calidad.
NotebookLM

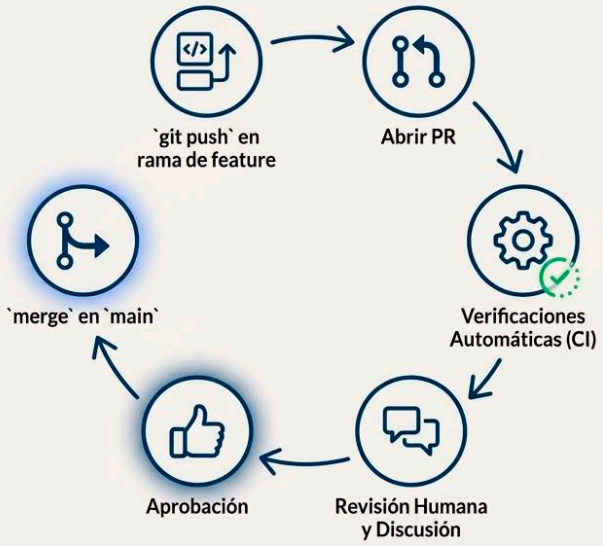
El Guardián de la Calidad: El Flujo del Pull Request (PR)

Definición

Una solicitud formal para integrar cambios en una rama protegida (`main`). Es el punto de control esencial en el desarrollo colaborativo.

Beneficios Key

Revisión de Código: Permite que otros revisen los cambios antes de la integración.
Mantenimiento de Calidad: Asegura consistencia y previene bugs.
Discusión y Documentación: Crea un registro de las decisiones de diseño.
Activación de CI/CD: Dispara tests automáticos y builds antes del merge.



NotebookLM

Los Principios de la Armonía

Commits Atómicos:
Un commit = una sola cosa lógica.

Mensajes Útiles:
Usa **Conventional Commits** para dar semántica (`feat`, `fix`, `refactor`).

`main` es Sagrado:
Nunca hagas commit directamente. Crea siempre una rama.

Ramas Efímeras:
Cuanto más cortas, menor es la probabilidad de conflictos.

Sincronización Constante:
Usa `pull --rebase` frecuentemente para mantener tu rama actualizada.

La Comunicación es Clave:
Habla con tu equipo sobre ediciones en archivos 'calientes'.

NotebookLM


De Arquitecto a Maestro: La Colaboración como Arte
El dominio de Git no reside en memorizar comandos, sino en la habilidad de diseñar sistemas de colaboración. Pasar de *reaccionar* al caos a diseñar*flujos de trabajo armoniosos es el verdadero camino hacia la maestría.
NotebookLM